CMSC 420: Amortized Analysis

Justin Wyss-Gallifent

January 30, 2026

1 Problems **2**

1 Problems

1. Aggregate method: Suppose we perform n operations where each operation costs 1.

   (a) What is the cost $C(n)$ of performing n operations?
   Solution:
   The cost is:
   $$\sum_{i=1}^{n} 1 = n$$

   (b) What is the amortized per-operation cost of performing n operations?
   Solution:
   The amortized cost is:
   $$A(n) = \frac{C(n)}{n} = 1$$

2. Aggregate method: Suppose we perform n operations where the ith operation costs i.

   (a) What is the cost $C(n)$ of performing n operations?

   Solution:
   The cost is:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

   (b) What is the amortized per-operation cost of performing n operations?

   Solution:
   The amortized cost is:

$$A(n) = \frac{C(n)}{n} = \frac{n+1}{2}$$

3. Aggregate method: Suppose we perform n operations. Each operation costs 2 except every tenth operation costs an additional 14.

   (a) What is the cost $C(n)$ of performing n operations?
   Solution:
   The total cost will be:

$$C(n) = 2n + 14\left\lfloor \frac{n}{10} \right\rfloor$$

   (b) What is the amortized per-operation cost of performing n operations?
   Solution:
   We have:

$$AC(n) = \frac{C(n)}{n} = \frac{2n + 14\left\lfloor \frac{n}{10} \right\rfloor}{n} = 2 + \frac{14\left\lfloor \frac{n}{10} \right\rfloor}{n}$$

   (c) Prove that the amortized cost is $\mathcal{O}(1)$.
   Solution:
   We have:

$$AC(n) = 2 + \frac{14\left\lfloor \frac{n}{10} \right\rfloor}{n} \leq 2 + \frac{14(n/10)}{n} = \frac{17}{5}$$

   The result follows.

4. Aggregate method: Suppose we perform n operations. The ith operation costs $2i$ except every tenth operation costs an additional i (so the $i = 10$ operation costs an extra 10, the $i = 20$ operation costs an extra 20, and so on).

   (a) What is the cost $C(n)$ of performing n operations?
   Solution:
   The base cost (without the additional) will be:

$$\sum_{i=1}^{n} 2i = n(n+1)$$

   The additional cost will be:

$$10 + 20 + 30 + ... + 10k$$

   Here k is the largest integer such that $10k \leq n$, so $k = \lfloor n/10 \rfloor$. Thus the additional cost will be:

$$10+20+30+...+\lfloor n/10 \rfloor 10 = 10\left(\frac{\lfloor n/10 \rfloor (\lfloor n/10 \rfloor + 1}{2}\right) = 5\lfloor n/10 \rfloor (\lfloor n/10 \rfloor + 1)$$

   Thus the total cost is:

$$C(n) = n(n+1) + 5\lfloor n/10 \rfloor (\lfloor n/10 \rfloor + 1)$$

   (b) What is the amortized per-operation cost of performing n operations?
   Solution:
   We have:

$$AC(n) = \frac{C(n)}{n} = \frac{n(n+1) + 5\lfloor n/10 \rfloor (\lfloor n/10 \rfloor + 1)}{n}$$

   (c) Prove that the amortized cost is $\mathcal{O}(n)$.
   Solution:
   We have:

$$\begin{aligned} AC(n) &= \frac{n(n+1) + 5\lfloor n/10 \rfloor (\lfloor n/10 \rfloor + 1)}{n} \\ &\leq (n+1) + 5(n/10)(n/10+1)/n \\ &\leq (n+1) + 5(0.01n^2 + 0.1n)/n \\ &\leq (n+1) + 5(0.01n + 0.1) \end{aligned}$$

   The result follows.

5. Token method: Suppose we execute a sequence of n operations on a data structure. Each operation has a base cost of 2 but every tenth operation has a supplemental cost of 4. Calculate the amortized cost. You can focus on a generalized run.

   Solution:
   Suppose we deposit β tokens per operation. One run is a sequence of 10 operations so during the run we deposit 10β tokens. Each of the first 9 operations costs 2 and the tenth costs $2 + 4 = 6$ for a total cost of $9(2) + 6 = 24$.
   Thus we wish to find the smallest β with $10\beta \geq 24$. This solves to $\beta \geq \frac{24}{10} = \frac{12}{5}$.
   Clearly $\beta = \frac{12}{5}$ satisfies this.

6. Token method: Suppose we are allocating memory for a stack. Each time a push would overflow the allocated memory we first reallocate to thrice the previous length, copy over the original stack, then do the push. The reallocation cost in tokens equals the new space allocated and this includes the copying of the old stack. Each push costs 1 token. Calculate the amortized cost. You can focus on a generalized run.

   Solution:
   For each $k \geq 1$, a run...

   - ...starts when we overflow $3k$ and reallocate to $9k$. There are $3k+1$ bytes on the stack.
   - ...ends when we overflow $9k$ and reallocate to $27k$. There are $9k+1$ bytes on the stack.

   (Note that since we pushed n items and we are focusing on a run within that process we know that $9k+1 \leq n$. We won't need this but it's worth keeping in mind for other problems.)

   Thus we have pushed $9k+1-(3k+1) = 6k$ items and we reallocated to $27k$ for a total cost of:

   $$6k + 27k = 33k$$

   Since the run involved $6k$ pushes we have deposited $\beta(6k)$ tokens and thus we need the smallest β such that:

   $$\beta(6k) \geq 33k \quad \text{for each } k \geq 1$$

   Notice that the k cancel and we get:

   $$\beta \geq \frac{33}{6} = \frac{11}{2}$$

   The smallest possible β is thus $\beta = 11/2$.

7. Token method: We insert n keys into an empty binary search tree at cost of 1 per insertion. When $n = 2^i$ for any $i \geq 0$ we rebuild the entire tree (it doesn't matter how) at a cost of 4^i. Prove that the amortized cost is $\mathcal{O}(n)$. You can focus on a generalized run.

   Solution:
   For each $k \geq 0$, a run...

   - ...starts after a rebuild, so there are 2^k keys in the tree.
   - ...ends after the next rebuild, so there are 2^{k+1} keys in the tree.

   (Note that since we pushed n items and we are focusing on a run within that process we know that $2^{k+1} \leq n$. We will need this.)

   Thus we have inserted $2^{k+1} - 2^k = 2^k$ keys and we rebuilt the tree at a cost of 4^{k+1} for a total cost of:

$$2^k + 4^{k+1}$$

   Since the run involved 2^k insertions we have deposited $\beta(2^k)$ tokens and thus we need the smallest β such that:

$$\beta(2^k) \geq 2^k + 4^{k+1}$$

   We solve this, keeping in mind that we are looking for the smallest possible β satisfying:

$$\begin{aligned}
\beta(2^k) &\geq 2^k + 4^{k+1} \\
\beta &\geq 1 + \frac{4 \cdot 4^k}{2^k} \\
\beta &\geq 1 + 4\left(\frac{4}{2}\right)^k \\
\beta &\geq 1 + 2^{k+2}
\end{aligned}$$

   Now then since $2^{k+1} \leq n$ we know that $2^{k+2} \leq 2n$. Thus assigning $\beta = 1 + 2n$ will suffice for all relevant k.
   Thus the amortized cost is $\mathcal{O}(n)$.

8. Both methods: Consider amortized cost for the scenario where we have a stack such that:

   - A pop costs 1.
   - A push costs 1, not counting any allocation that may be necessary.
   - Re-allocation costs the new size. This cost includes copying the contents of the old stack.

   Assume we start with 0 bytes allocated and when we re-allocate we multiply the length by $\alpha \in \mathcal{Z}$ where $\alpha \geq 2$ except for the very first re-allocation which goes from 0 to 1.

   Define:

$$f(\alpha) = \frac{\alpha^2 + \alpha - 1}{\alpha - 1}$$

   (a) Use the aggregate method to prove that in the worst case with n operations the amortized cost $AC(n)$ of each stack operation satisfies $AC(n) \leq f(\alpha)$.

   Solution:

   In a worst-case scenario we just push n times. In such a case we have push costs $1 + 1 + ... + 1$ (n times) and reallocation costs $1 + \alpha + \alpha^2 + ... + \alpha^k$ with the property that k is the smallest integer with $\alpha^k \geq n$. This is equivalent to $k \geq \log_\alpha n$ so $k = \lceil \log_\alpha n \rceil$.

   Thus the total cost will satisfy:

$$\begin{aligned}
C(n) &\leq n + 1 + \alpha + \alpha^2 + ... + \alpha^{\lceil \log_\alpha n \rceil} \\
&\leq n + \frac{\alpha^{\lceil \log_\alpha n \rceil + 1} - 1}{\alpha - 1} \\
&\leq n + \frac{\alpha^{2 + \log_\alpha n} - 1}{\alpha - 1} \\
&\leq n + \frac{\alpha^2 \cdot \alpha^{\log_\alpha n} - 1}{\alpha - 1} \\
&\leq n + \frac{\alpha^2 n - 1}{\alpha - 1} \\
&\leq \frac{n(\alpha - 1) + \alpha^2 n - 1}{\alpha - 1} \\
&\leq \frac{\alpha^2 n + \alpha n - n - 1}{\alpha - 1} \\
&\leq \frac{\alpha^2 n + \alpha n - n}{\alpha - 1} - \frac{1}{\alpha - 1} \\
&\leq \frac{\alpha^2 n + \alpha n - n}{\alpha - 1}
\end{aligned}$$

The amortized cost will then satisfy:

$$AC(n) \leq \frac{C}{n} \leq \frac{\alpha^2 + \alpha - 1}{\alpha - 1}$$

(b) Prove mathematically that this expression is minimum when $\alpha = 2$.

Solution:

Observe that:

$$\frac{\alpha^2 + \alpha - 1}{\alpha - 1} = \alpha + 2 + \frac{1}{\alpha - 1}$$

So:

$$\frac{df}{d\alpha} = 1 - \frac{1}{(\alpha - 1)^2}$$

This expression is positive so $f(\alpha)$ is increasing, hence the minimum is at $\alpha = 2$.

(c) Use the token method to prove that collecting $f(\alpha)$ tokens per push/pop is sufficient. As in class you can ignore the first and last runs.

Solution:

Suppose we have just reallocated from length k to length αk and done a push, meaning there are $k + 1$ elements on the stack. The next reallocation will occur at length αk, when we reallocate to $\alpha^2 k$ and do a push to $\alpha k + 1$ elements. Before that happens we will get at least $(\alpha - 1)k - 1$ cheap pushes and pops and then one final push over the edge, which is expensive.

The cheap pushes and pops cover their own costs since we collect $\frac{\alpha^2+\alpha-1}{\alpha-1}$ tokens each but only spend 1, meaning at the end, right before the reallocation plus push we have at least

$$\left(\frac{\alpha^2 + \alpha - 1}{\alpha - 1} - 1\right)((\alpha - 1)k - 1)$$

tokens in the bank.

The final push, not counting reallocation, collects $\frac{\alpha^2+\alpha-1}{\alpha-1}$ tokens as well and spends 1 on pushing, meaning we have

$$\left(\frac{\alpha^2 + \alpha - 1}{\alpha - 1} - 1\right)(\alpha - 1)k = (\alpha^2 + \alpha - 1 - (\alpha - 1))k = \alpha^2 k$$

tokens in the bank.

However the reallocation to $\alpha^2 k$ costs exactly this amount, and we have it covered!

9. Aggregate method: Consider amortized cost for the stack allocation problem where:

   - A pop costs 1.
   - A push costs 1, not counting any allocation that may be necessary.
   - When reallocation is necessary we multiply the length by 3.
   - Reallocation costs the size of the new list and includes copying the old elements over.
   - The list starts with one space allocated and nothing in it.

   Suppose that we do some sequence of n pushes and pops. Use the aggregate method to prove that the amortized cost is less than 5.5.

   Solution:
   The push costs are $1 + 1 + ... + 1$ a total of n times.
   The reallocation costs are $3 + 9 + 27 + ... + 3^k$ which go until the smallest k such that $3^k \geq n$. This will be $k \geq \log_3 n$ and so $k = \lceil \log_3 n \rceil$.
   The total cost is then:

$$
\begin{aligned}
C &= n + \sum_{i=1}^{\lceil \log_3 n \rceil} 3^i \\
&\leq n + \sum_{i=1}^{1+\log_3 n} 3^i \\
&\leq n + \frac{3^{2+\log_3 n} - 1}{3-1} - 1 \\
&\leq n + \frac{9}{2} \cdot 3^{\log_3 n} - \frac{1}{2} - 1 \\
&\leq n + \frac{9}{2} n - \frac{3}{2} \\
&\leq \frac{11}{2} n - \frac{3}{2}
\end{aligned}
$$

   The amorized cost then satisfies:

$$
AC = \frac{C}{n} \leq \frac{11}{2} - \frac{3/2}{n} \leq 5.5
$$

10. Aggregate method: Suppose you are working with a stack allocation onto which you will push n items.

    You start with zero entries allocated and decide that when a push will overflow the list you will first reallocate to the next highest perfect square, copy over the old stack entries, then push.

    For example the $i = 1$ append overflows and hence requires a reallocation to length 1. The $i = 2$ append overflows and hence requires a reallocation to length 4. The $i = 3, 4$ appends require no reallocation but the $i = 5$ append overflows and hence requires a reallocation to length 9. And so on.

    Assuming that each append cost 1 and reallocating costs the size of the new list, including the copying, prove that the amortized cost is $\mathcal{O}(\sqrt{n})$.

    Use the aggregate method.

    Solution:
    We push n items which costs n. The reallocations cost:

$$1 + 2 + 4 + ... + k^2$$

    The value of k is the smallest integer with $k^2 \geq n$ and hence $k = \sqrt{n}$ which will be $k = \lceil \sqrt{n} \rceil$. Consequently the total cost is:

$$\begin{aligned} C(n) &= n + \sum_{i=1}^{\lceil \sqrt{n} \rceil} i^2 \\ &= n + \frac{\lceil \sqrt{n} \rceil (\lceil \sqrt{n} \rceil + 1)(2 \lceil \sqrt{n} \rceil + 1)}{6} \\ &\leq n + \frac{(\sqrt{n} + 1)(\sqrt{n} + 1 + 1)(2(\sqrt{n} + 1) + 1)}{6} \end{aligned}$$

    Thus $C(n) = \mathcal{O}(n^{3/2})$ and since $AC(n) = C(n)/n$ we have $AC(n) = \mathcal{O}(\sqrt{n})$.

11. Aggregate method: Suppose we have an list A which represents a binary string, so $A[0]$ represents the $2^0 = 1$s digit, $A[1]$ represents the $2^1 = 2$s digit, $A[2]$ represents the $2^2 = 4$s digit, and so on.

    Suppose A starts at all 0s and we then implement n increment operations. Every time a bit flips the cost is 2^k where 2^k is the index of the bit.

    Show that $AC(n) = \mathcal{O}(\lg n)$.

    Solution:
    Let's calculate the corresponding amortized cost using the aggregate method. Observe that every increment flips $A[0]$, every second increment flips $A[1]$ as well, every fourth increment flips $A[2]$ as well, and so on.
    This means that if we implement n increments $A[0]$ will flip n times, $A[1]$ will flip $\lfloor n/2 \rfloor$ times, $A[2]$ will flip $\lfloor n/4 \rfloor$ times, and so on, for a total cost of:

$$C(n) = n(2^0) + \lfloor n/2 \rfloor (2^1) + \lfloor n/4 \rfloor (2^2) + ... = \sum_{k=0}^{\infty} \lfloor n/2^k \rfloor (2^k)$$

    This sum is not infinite because each floor term is nonzero only when $n/2^k \geq 1$ and this happens for $k \leq \lg n$, which for integers is $k \leq \lfloor \lg n \rfloor$, thus the total cost is really:

$$C(n) = \sum_{k=0}^{\lfloor \lg n \rfloor} \lfloor n/2^k \rfloor (2^k)$$

    This is a bit of an awkward sum because of the inner floor function so we'll bound it:

$$\begin{aligned} C(n) &= \sum_{k=0}^{\lfloor \lg n \rfloor} \lfloor n/2^k \rfloor (2^k) \\ &\leq \sum_{k=0}^{\lfloor \lg n \rfloor} (n/2^k)(2^k) \\ &\leq n \sum_{k=0}^{\lfloor \lg n \rfloor} 1 \\ &\leq n \lfloor \lg n \rfloor \\ &\leq n \lg n \end{aligned}$$

    The amortized cost then satisfies:

$$AC(n) \leq \lg n$$

    Thus $AC(n) = \mathcal{O}(\lg n)$.

12. Token method: Revisiting the stack allocation problem suppose that when reallocation is necessary we multiply the length by some constant $C \in \mathcal{Z}$ with $C > 1$. Each push costs 1 and a reallocation cost equals the number of bytes being allocated and this includes copying the old data.

    (a) Use the token method to find an upper bound on the amortized cost. You can focus on a generalized run.

    Solution:
    A run ends when we have reallocated from length k to length Ck and there are $k+1$ bytes on the stack. Our next reallocation will be when we push one byte beyond Ck and we reallocate to length C^2k. Before that reallocation we will have $Ck-(k+1)$ cheap pushes which yield a surplus of:

    $$\beta(Ck-k-1)-(Ck-k-1)$$

    For the final push we gain another surplus of $\beta-1$ so our total surplus is now:

    $$\beta(Ck-k-1)-(Ck-k-1)+\beta-1$$

    To cover the final reallocation we then need:

    $$\begin{aligned}
    \beta(Ck-k-1)-(Ck-k-1)+\beta-1 &\geq C^2k \\
    \beta Ck-\beta k-\beta-Ck+k+1+\beta-1 &\geq C^2k \\
    \beta C-\beta-C+1 &\geq C^2 \\
    (\beta-1)(C-1) &\geq C^2 \\
    \beta &\geq \frac{C^2}{C-1}+1
    \end{aligned}$$

    The smallest such β is then exactly that, so:

    $$AC(n)=\frac{C^2}{C-1}+1$$

    (b) As $C \to \infty$, what happens to $AC(n)$?

    Solution:
    It approaches infinity.

13. Aggregate method: Consider amortized cost for the scenario where:

- We start with one position allocated but empty so a first push will simply fill that position and not require reallocation.
- A pop costs 1.
- A push costs 1, not counting any allocation that may be necessary.
- When reallocation is necessary we multiply the length by 5.
- Reallocation costs the size of the new list and includes copying the old elements over.

Suppose that we do some sequence of n pushes and pops. Use the aggregate method to prove that the (worst-case) amortized cost is less than 7.5.

Solution:
In the worst-case we have only pushes in order to reach reallocation as quickly as possible, hence n of them. The push costs are then $1+1+...+1 = n$.
The reallocation costs are $5 + 5^2 + 5^3 + ... + 5^k$ which go until the smallest k such that $5^k \geq n$. This will be $k \geq \log_5 n$ and so $k = \lceil \log_5 n \rceil$.
The total cost is then:

$$\begin{aligned}
C &= n + \sum_{i=1}^{\lceil \log_5 n \rceil} 5^i \\
&\leq n + \sum_{i=1}^{1+\log_5 n} 5^i \\
&\leq n + \frac{5^{2+\log_5 n} - 1}{5-1} - 1 \\
&\leq n + \frac{25}{4} \cdot 5^{\log_5 n} - \frac{1}{4} - 1 \\
&\leq n + \frac{25}{4} n - \frac{5}{4} \\
&\leq \frac{29}{4} n - \frac{5}{4}
\end{aligned}$$

The amortized cost then satisfies:

$$AC = \frac{C}{n} \leq \frac{29}{4} - \frac{5/4}{n} \leq 7.5$$

14. Aggregate method: Suppose we are working on a stack onto which we will push n items.

We start with zero entries allocated and decide that when a push will overflow the list we will reallocate to the next highest perfect cube length.

For example the $i = 1$ push overflows and hence requires a reallocation to length 1. The $i = 2$ push overflows and hence requires a reallocation to length 8. The $i = 3, 4, 5, 6, 7, 8$ pushes require no reallocation but the $i = 9$ append overflows and hence requires a reallocation to length 27. And so on.

Assuming that each push costs 1 and reallocating costs the size of the new list, including the copying, prove that the worst-case amortized cost is $\mathcal{O}\left(\sqrt[3]{n}\right)$.

Solution:
We push n items which costs n. The reallocations cost:

$$1 + 8 + 27 + ... + k^3$$

The value of k is the smallest integer with $k^3 \geq n$ and hence $k \geq \sqrt[3]{n}$ which will be $k = \lceil \sqrt[3]{n} \rceil$. Consequently the total cost is:

$$\begin{aligned} C(n) &= n + \sum_{i=1}^{\lceil \sqrt[3]{n} \rceil} i^3 \\ &= n + \frac{\lceil \sqrt[3]{n} \rceil^2 \left(\lceil \sqrt[3]{n} \rceil + 1\right)^2}{4} \\ &\leq n + \frac{\left(\sqrt[3]{n} + 1\right)^2 \left(\sqrt[3]{n} + 2\right)^2}{4} \end{aligned}$$

Thus $C(n) = \mathcal{O}(n^{4/3})$ and since $AC(n) = C(n)/n$ we have $AC(n) = \mathcal{O}\left(\sqrt[3]{n}\right)$.

15. Token method: Suppose we are working on a stack onto which we will push n items.

We start with zero entries allocated and decide that when a push will overflow the list we will reallocate to the next highest perfect cube length. For example the $i = 1$ push overflows and hence requires a reallocation to length 1. The $i = 2$ push overflows and hence requires a reallocation to length 8. The $i = 3, 4, 5, 6, 7, 8$ pushes require no reallocation but the $i = 9$ append overflows and hence requires a reallocation to length 27. And so on.

Assuming that each push costs 1 and reallocating costs the size of the new list, including the copying, prove that the worst-case amortized cost is $\mathcal{O}\left(\sqrt[3]{n}\right)$.

You can focus on a generalized run.

Solution:
We push n entries. A run starts after we have reallocated to length k where $k = j^3$ for some j and ends after we have reallocated to length $(j+1)^3$. Since we pushed n entries we have $(j+1)^3 \leq n$.
During that run the number of pushes is:

$$(j+1)^3 - j^3 = 3j^2 + 3j + 1$$

This is then the total push cost. At the end, the reallocation cost is just:

$$(j+1)^3$$

Thus the total cost is:

$$3j^2 + 3j + 1 + (j+1)^3 = j^3 + 6j^2 + 6j + 2$$

If we deposit β tokens per push then we have deposited a total of:

$$\beta(3j^2 + 3j + 1)$$

Consequently we want the β with:

$$\beta(3j^2 + 3j + 1) \geq j^3 + 6j^2 + 6j + 2$$
$$\beta \geq \frac{1}{3}j + \frac{5}{3} + \frac{\frac{2}{3}j + \frac{1}{3}}{3j^2 + 3j + 1}$$

The fractional part is less than or equal to 1 for all j (graphing is sufficient since it's positive and decreasing asymptotically to 0) and so it suffices to have:

$$\beta \geq \frac{1}{3}j + \frac{5}{3} + 1$$

This must be true for all j with $(j+1)^3 \leq n$ in other words for all j with $j \leq \sqrt[3]{n} - 1$. Thus it suffices to have:

$$\beta \geq \frac{1}{3}\left(\sqrt[3]{n} - 1\right) + \frac{8}{3}$$

Since we want the smallest such β it suffices to have:

$$\beta = \frac{1}{3}\left(\sqrt[3]{n} - 1\right) + \frac{8}{3} = \mathcal{O}(\sqrt[3]{n})$$

16. Aggregate method: We insert n keys into an empty binary search tree at cost of 1 per insertion. When $n = 2^i$ for any $i \geq 0$ we rebuild the entire tree (it doesn't matter how) at a cost of 4^i.

(a) Find an expression for the exact cost $C(n)$ of inserting the n keys.

Solution:

The costs of n insertions is $1 + 1 + ... + 1 = n$. The rebuilds cost:

$$4^0 + 4^1 + 4^2 + 4^3 + ... + 4^k$$

Here k is the largest integer with $2^k \leq n$ so $k = \lfloor \lg n \rfloor$. Thus the total cost is:

$$\begin{aligned} C(n) &= n + \sum_{i=0}^{\lfloor \lg n \rfloor} 4^i \\ &= n + \frac{4^{\lfloor \lg n \rfloor + 1} - 1}{4 - 1} \\ &= n + \frac{1}{3}\left(4^{\lfloor \lg n \rfloor + 1} - 1\right) \end{aligned}$$

(b) It is a fact that $C(n) < n + 4^{\lfloor \lg n \rfloor + 1}$ (this is given in case you make a mistake in (a)). Use this fact to prove that $AC(n) = \mathcal{O}(n)$.

Solution:

We have:

$$\begin{aligned} C(n) &< n + 4^{\lfloor \lg n \rfloor + 1} \\ &< n + 4^{\lg(n)+1} \\ &< n + 2^{2(\lg(n)+1)} \\ &< n + 2^{2\lg(n)+2} \\ &< n + 2^2 \cdot 2^{\lg(n^2)} \\ &< n + 4n^2 \end{aligned}$$

Consequently:

$$AC(n) = \frac{C(n)}{n} < 1 + 4n$$

Thus we have our claim.

17. Suppose you are doing the aggregate method of amortized analysis and you find that:

$$C(n) = 8^{\lceil \lg n \rceil} - 1$$

Prove that $AC(n) = \mathcal{O}(n^k)$ for an appropriate integer k.

Solution:

Observe that:

$$\begin{aligned} AC(n) &= \frac{8^{\lceil \lg n \rceil} - 1}{n} \\ &\leq \frac{8^{1+\lg n}}{n} \\ &\leq \frac{2^{3+3\lg n}}{n} \\ &\leq \frac{8n^3}{n} \\ &\leq 8n^2 \end{aligned}$$

Thus $AC(n) = \mathcal{O}(n^2)$.

18. Suppose we insert n keys into a binary search tree which starts empty. Each insertion costs 7 but whenever $n = 2^i - 1$ for each $i \geq 1$ we rebuild the binary tree (it doesn't matter how) at a cost of n; so we rebuild at each of $n = 1, 3, 7, 15, ...$ at a cost of $1, 3, 7, 15, ...$ respectively. Calculate the total exact cost $C(n)$ for performing these n insertions. Show all work and justify (at least a few words!) where any floors and/or ceilings come from. You should evaluate any resulting sums but you don't need to simplify or bound the result or find $AC(n)$.

Solution:

The base cost is $7n$.
The total rebuilding cost is:

$$1 + 3 + 7 + ... + (2^k - 1)$$

where k is chosen to be the largest value with $2^k - 1 \leq n$, so $2^k \leq n + 1$ and so $k \leq \lg(n + 1)$. Thus $k = \lfloor \lg(n + 1) \rfloor$. Thus the total cost is:

$$\begin{aligned} C(n) &= 7n + \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} (2^i - 1) \\ &= 7n + \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} 2^i - \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} 1 \\ &= 7n + 2^{\lfloor \lg(n+1) \rfloor + 1} - 1 - 1 - \lfloor \lg(n + 1) \rfloor \end{aligned}$$

19. Suppose we insert n keys into a binary search tree which starts empty. Each insertion costs 1 but whenever $n = 2^i - 1$ for each $i \geq 1$ we rebuild the binary tree (it doesn't matter how) at a cost of n; so we rebuild at each of $n = 1, 3, 7, 15, ...$ at a cost of $1, 3, 7, 15, ...$ respectively.

Using the token method we say a run starts after a rebuild has just occurred and ends after the next rebuild so for each $k \geq 0$ a run goes from $2^k - 1$ to $2^{k+1} - 1$ keys.

(a) If we deposit β tokens per operation, how many tokens will we deposit during a general (k is unknown) run?

Deposit	$\beta \cdot 2^k$

(b) What will the cost of a general (k is unknown) run be? Simplify.

Cost	$2^k + 2^{k+1} - 1$

(c) What it the smallest β such that we will deposit enough to cover the costs? Show your work.

Solution:

We need:

$$\beta \cdot 2^k \geq 2^k + 2^{k+1} - 1$$
$$\beta \geq 1 + 2 - \frac{1}{2^k}$$

Clearly if $\beta = 3$ then this will be satisfied.